SprintSync — AI-Powered Agile Project Management Platform

Full-Stack SaaS Application | MERN Stack | SDE2-Level Architecture

---

Table of Contents

---

1. Executive Summary

SprintSync is an AI-first, real-time agile project management SaaS platform built on the MERN stack. It targets software development teams and product managers who find Jira overly complex and Notion too flexible. The platform uses artificial intelligence to automate sprint planning, predict task completion times, identify potential blockers before they occur, and generate standup summaries — all while maintaining the simplicity of a modern kanban tool.

Key Differentiators:

- AI-generated sprint plans from natural language descriptions
- Predictive blocker detection based on task dependencies and historical patterns
- Real-time collaborative board with live cursors and presence tracking
- Transparent, per-seat pricing with no hidden enterprise costs
- Native integrations with GitHub, GitLab, Slack, and calendar systems

Target Market:

- Small-to-medium software teams (5-50 developers)

- Remote-first companies
- Product agencies managing multiple client projects
- Open-source projects needing lightweight coordination

---

2. Problem Statement

The Current Landscape

Modern software teams face a critical tooling gap in project management:

Tool	Strength	Critical Weakness
Jira	Enterprise feature depth	Overwhelming complexity, steep learning curve, slow performance
Notion	Flexibility & documents	No native agile workflows, manual sprint management, no AI assistance
Trello	Simplicity	Lacks sprint planning, reporting, and enterprise features
Linear	Speed & design	Limited AI capabilities, no predictive analytics
Asana	General project mgmt	Not built for software development workflows

Pain Points Identified

1. **Sprint Planning Overhead**: Teams spend 2-4 hours per sprint in planning meetings manually breaking down epics, estimating story points, and assigning tasks.
2. **Invisible Blockers**: Dependencies between tasks are often discovered mid-sprint, causing delays that could have been predicted.
3. **Remote Team Disconnect**: Async teams lack visibility into who is working on what in real-time, leading to duplicate work and communication gaps.
4. **Tool Fatigue**: Teams use 5-7 separate tools (Jira + Slack + GitHub + Google Docs + Zoom + Loom + Confluence) with no unified workflow.
5. **Enterprise Pricing Trap**: Traditional tools lock critical features behind opaque "Contact Sales" enterprise tiers.

---

3. Solution Overview

Core Value Proposition

> "Describe what you want to build. SprintSync plans your sprint, predicts your blockers, and keeps your team synchronized — automatically."

Key Features

3.1 AI Sprint Planning

- Input a product goal in natural language (e.g., *"Build OAuth2 authentication with Google and GitHub, including JWT refresh tokens and MFA support"*)

- AI generates a complete sprint with:
  - Decomposed user stories and tasks
  - Story point estimates (Fibonacci scale)
  - Optimal task sequencing based on dependencies
  - Balanced workload distribution across team members
  - 20% buffer allocation for bugs and technical debt

3.2 Predictive Blocker Detection

- Analyzes task dependencies, assignee workload, and historical sprint data
- Flags potential blockers before the sprint begins
- Suggests mitigation strategies (reassign tasks, split stories, extend sprint)
- Confidence scoring for each prediction

3.3 Real-Time Collaborative Board

- Live cursor tracking on kanban boards
- Instant task status updates across all clients
- Presence indicators (active, away, offline)
- Comment threads with @mentions and real-time typing indicators
- Conflict-free concurrent editing of task descriptions

3.4 Automated Standups

- AI-generated daily standup summaries based on:
  - Code commits (GitHub/GitLab integration)
  - Task movements on the board
  - Time tracking data
  - Manual check-ins
- Delivered via Slack, email, or in-app dashboard

3.5 Analytics & Reporting

- Burndown charts with ideal vs. actual vs. AI-predicted velocity
- Team productivity heatmaps
- Sprint retrospective insights (what went well, what blocked, velocity trends)
- DORA metrics integration (deployment frequency, lead time, MTTR, change failure rate)

---

4. Technology Stack

4.1 Backend

Technology	Purpose	Justification
Node.js 20 (LTS)	Runtime	Event-driven, non-blocking I/O ideal for real-time SaaS
Express.js 4	Web framework	Mature ecosystem, middleware-rich, proven at scale

Table 2 – continued

Technology	Purpose	Justification
TypeScript 5	Language	Type safety, better IDE support, reduced runtime errors
MongoDB 7	Primary database	Native document model fits nested project data; horizontal scaling via sharding
Mongoose 8	ODM	Schema validation, middleware hooks, query building
Redis 7	Cache & Pub/Sub	Session store, rate limiting, Socket.IO adapter, job queues
BullMQ	Job queue	Redis-backed, reliable, with retries, rate limiting, and observability
Socket.IO 4	Real-time communication	WebSocket with fallback, room-based broadcasting, middleware support
OpenAI API	AI engine	GPT-4o for complex reasoning, fine-tuned GPT-3.5 for estimation
Stripe	Payments	Industry standard, robust subscription management, excellent webhook system
JWT + bcrypt	Authentication	Stateless auth, password hashing, refresh token rotation
Winston + Pino	Logging	Structured logging with multiple transports
Jest + Supertest	Testing	Unit and integration testing with coverage reporting

4.2 Frontend

Technology	Purpose	Justification
React 19	UI library	Concurrent features, server components, optimal re-rendering
Vite 5	Build tool	Fast HMR, optimized production builds, native ESM
TypeScript 5	Language	Type-safe components, prop interfaces, reduced bugs
Zustand	Global state	Minimal boilerplate, excellent TypeScript support, middleware ecosystem
React Query (TanStack Query)	Server state	Caching, background refetching, optimistic updates, deduplication
React Router 6	Routing	Nested routes, lazy loading, route guards, data loaders
Socket.IO Client	Real-time	Seamless WebSocket integration with React hooks
Recharts + D3.js	Data visualization	Declarative React charts + custom D3 for complex visualizations

Table 3 – continued

Technology	Purpose	Justification
Tailwind CSS 3	Styling	Utility-first, rapid development, consistent design system
shadcn/ui	Component library	Accessible, customizable, built on Radix UI primitives
React Hook Form + Zod	Forms & validation	Performant forms with schema-based validation
Framer Motion	Animations	Declarative animations for board interactions and page transitions

4.3 DevOps & Infrastructure

Technology	Purpose
Docker	Containerization
Docker Compose	Local development orchestration
Nginx	Reverse proxy, SSL termination, static asset serving
Kubernetes	Production orchestration (future)
GitHub Actions	CI/CD pipelines
AWS S3 + CloudFront	File storage and CDN
Prometheus + Grafana	Monitoring and alerting
Sentry	Error tracking and performance monitoring

---

5. System Architecture

5.1 High-Level Architecture

```
Client Layer
  - React 19 (Vite)
  - WebSocket Client
  - React Query + Zustand
       |
       v
Load Balancer (Nginx)
  - SSL Termination
  - Rate Limiting
  - Static Assets
       |
       v
API Gateway Layer
  - Express Server (x3 instances)
  - Socket Server (x2 instances)
  - BullMQ Workers
       |
       v
```

```
Service Layer (Modular)
  - Auth Service
  - Sprint Service
  - Task Service
  - AI Service
  - Billing Service
  - Notification Service
  - Analytics Service
  - Search Service
       |
       v
Data Layer
  - MongoDB (Primary + Replica)
  - Redis (Session + Pub/Sub)
  - AWS S3 + CDN (File Storage)
       |
       v
External Services
  - Stripe (Billing)
  - OpenAI (AI/GPT)
  - SendGrid/AWS SES (Email)
  - Slack (Webhooks)
  - GitHub (OAuth)
  - Sentry/Datadog (Monitoring)
```

5.2 Architectural Decisions

Monolithic Modular vs. Microservices Decision: Start with a modular monolith, extract to microservices when hitting $1M ARR.

Rationale:

- Microservices add 3-6 months of infrastructure overhead
- A single team (1-3 developers) cannot effectively manage distributed systems
- Network latency between services would hurt real-time performance
- MongoDB and Node.js can scale vertically and horizontally within a monolith
- Clear module boundaries (`auth/`, `sprint/`, `task/`, `ai/`, `billing/`) allow future extraction

MongoDB vs. PostgreSQL Decision: MongoDB as primary database.

Rationale:

- Project management data is inherently document-oriented (nested tasks, flexible schemas)
- Activity logs and event streams fit MongoDB's time-series patterns
- Embedded subdocuments reduce JOIN complexity vs. relational models
- Native horizontal scaling via sharding for multi-tenant SaaS
- GridFS for receipt/attachment storage

Redis Pub/Sub vs. Apache Kafka **Decision:** Redis Pub/Sub for Socket.IO adapter and event bus.

Rationale:

- Redis Pub/Sub handles 100K+ messages/second, sufficient for MVP and growth phases
- Lower operational complexity than Kafka (no Zookeeper, no topic management)
- Socket.IO has a first-party Redis adapter
- Can migrate to Kafka/NATS when concurrent connections exceed 100K

---

6. Database Design

6.1 Collection Schema

workspaces

```
{
  _id: ObjectId,
  name: String,                    // "Acme Engineering"
  slug: String,                    // "acme-engineering" (indexed, unique)
  ownerId: ObjectId,               // Reference to users
  plan: String,                    // Enum: ['free', 'pro', 'enterprise']

  // Billing
  stripeCustomerId: String,
  stripeSubscriptionId: String,
  billingEmail: String,

  // Settings
  settings: {
    defaultSprintDuration: Number, // Days, default 14
    workingDays: [Number],          // [1,2,3,4,5] = Mon-Fri
    storyPointScale: [Number],      // [1, 2, 3, 5, 8, 13]
    timeZone: String,               // "America/New_York"
    dateFormat: String              // "MM/DD/YYYY"
  },

  // Members
  memberIds: [ObjectId],           // References to users
  inviteCodes: [{
    code: String,
    role: String,                  // 'admin', 'manager', 'member'
    expiresAt: Date,
    usedBy: ObjectId
  }],

  // Integrations
  integrations: {
    slack: { webhookUrl: String, channel: String },
    github: { installationId: Number, repositories: [String] },
    gitlab: { accessToken: String, projectIds: [Number] }
```

```
  },

  // Quotas (enforced by middleware)
  quotas: {
    maxProjects: Number,
    maxMembers: Number,
    aiGenerationsPerMonth: Number,
    storageGb: Number
  },

  createdAt: Date,
  updatedAt: Date
}

// Indexes
// { slug: 1 } — unique
// { ownerId: 1 }
// { memberIds: 1 }
// { stripeCustomerId: 1 }
```

users

```
{
  _id: ObjectId,
  email: String,                    // Indexed, unique
  passwordHash: String,             // bcrypt, null for OAuth users

  // Profile
  profile: {
    firstName: String,
    lastName: String,
    avatarUrl: String,
    timezone: String,
    notificationPreferences: {
      email: Boolean,
      push: Boolean,
      slack: Boolean,
      dailyDigest: Boolean
    }
  },

  // Auth
  auth: {
    provider: String,               // 'local', 'google', 'github', 'microsoft'
    providerId: String,             // OAuth provider user ID
    mfaEnabled: Boolean,
    mfaSecret: String,              // Encrypted TOTP secret
    emailVerified: Boolean,
    lastLoginAt: Date,
    lastLoginIp: String
  },
```

```
  // Security
  security: {
    failedLoginAttempts: Number,
    lockedUntil: Date,
    passwordChangedAt: Date,
    previousPasswords: [String]     // Last 5 hashes
  },

  // Workspaces (denormalized for quick lookup)
  workspaces: [{
    workspaceId: ObjectId,
    role: String,                   // 'owner', 'admin', 'manager', 'member'
    joinedAt: Date
  }],

  createdAt: Date,
  updatedAt: Date
}

// Indexes
// { email: 1 } — unique, sparse
// { 'auth.providerId': 1, 'auth.provider': 1 } — unique, sparse
// { 'workspaces.workspaceId': 1 }
```

projects

```
{
  _id: ObjectId,
  workspaceId: ObjectId,            // Indexed
  name: String,
  description: String,
  key: String,                      // Short code, e.g., "WEB"
  status: String,                   // 'active', 'archived', 'deleted'

  // Members with project-level roles
  members: [{
    userId: ObjectId,
    role: String                    // 'lead', 'contributor', 'viewer'
  }],

  // Configuration
  config: {
    defaultAssignee: ObjectId,
    issueTypes: [String],           // ['bug', 'feature', 'task', 'epic']
    labels: [{ name: String, color: String }],
    workflows: [{
      name: String,
      statuses: [{
        name: String,
        category: String            // 'todo', 'in_progress', 'done'
```

```
    }]
  }]
 },

 createdBy: ObjectId,
 createdAt: Date,
 updatedAt: Date
}

// Indexes
// { workspaceId: 1, status: 1 }
// { key: 1, workspaceId: 1 } — unique
```

sprints

```
{
 _id: ObjectId,
 workspaceId: ObjectId,
 projectId: ObjectId,

 // Basic Info
 name: String,                    // "Sprint 24: Auth Overhaul"
 goal: String,                    // "Implement OAuth2 and MFA"
 status: String,                  // 'planning', 'active', 'completed', 'cancelled'

 // Timeline
 startDate: Date,
 endDate: Date,

 // Capacity & Progress
 totalPoints: Number,             // Sum of all task story points
 completedPoints: Number,

 // AI Metadata
 aiGenerated: Boolean,
 aiMetadata: {
  promptUsed: String,
  modelVersion: String,           // "gpt-4o-2024-08-06"
  confidenceScore: Number,        // 0.0 - 1.0
  generationTimeMs: Number
 },

 // Retrospective (populated after completion)
 retrospective: {
  whatWentWell: [String],
  whatToImprove: [String],
  actionItems: [String],
  velocity: Number,               // Actual velocity for this sprint
  predictedVelocity: Number       // AI prediction
 },
```

```
  createdBy: ObjectId,
  createdAt: Date,
  updatedAt: Date
}

// Indexes
// { workspaceId: 1, status: 1 }
// { projectId: 1, startDate: -1 }
// { startDate: 1, endDate: 1 }
```

tasks

```
{
  _id: ObjectId,
  workspaceId: ObjectId,             // Shard key for horizontal scaling
  projectId: ObjectId,
  sprintId: ObjectId,                // null if in backlog

  // Content
  title: String,
  description: String,               // Rich text (Markdown)

  // Classification
  type: String,                      // 'bug', 'feature', 'task', 'epic', 'subtask'
  status: String,                    // 'backlog', 'todo', 'in_progress', 'review', 'done'
  priority: String,                  // 'low', 'medium', 'high', 'urgent'

  // Estimation
  storyPoints: Number,               // Fibonacci: 1, 2, 3, 5, 8, 13
  timeEstimate: Number,              // Minutes
  timeSpent: Number,                 // Minutes (auto + manual)

  // AI Estimation
  aiEstimate: {
    suggestedPoints: Number,
    confidence: Number,              // 0.0 - 1.0
    reasoning: String                // "Similar to TASK-123 which took 5 points"
  },

  // People
  assigneeId: ObjectId,
  reporterId: ObjectId,

  // Relationships
  parentId: ObjectId,                // For subtasks
  subtaskIds: [ObjectId],
  dependencies: [{
    taskId: ObjectId,
    type: String,                    // 'blocks', 'is_blocked_by', 'relates_to'
    status: String                   // 'active', 'resolved'
  }],
```

```
  // Blockers
  blockers: [{
    description: String,
    predictedBy: String,            // 'ai' or 'user'
    predictedAt: Date,
    resolvedAt: Date,
    resolvedBy: ObjectId
  }],

  // Categorization
  labels: [String],
  epicId: ObjectId,

  // Attachments
  attachments: [{
    filename: String,
    s3Key: String,
    mimeType: String,
    size: Number,
    uploadedBy: ObjectId,
    uploadedAt: Date
  }],

  // Activity Log (embedded for fast reads, capped at 100 entries)
  activityLog: [{
    action: String,                 // 'created', 'status_changed', 'assigned', 'commented'
    actorId: ObjectId,
    timestamp: Date,
    metadata: Object,               // { from: 'todo', to: 'in_progress' }
    _id: ObjectId
  }],

  // Comments (embedded, capped at 50)
  comments: [{
    authorId: ObjectId,
    content: String,
    mentions: [ObjectId],
    createdAt: Date,
    updatedAt: Date,
    _id: ObjectId
  }],

  // Due dates
  dueDate: Date,
  completedAt: Date,

  createdAt: Date,
  updatedAt: Date
}
```

```
// Indexes
// { workspaceId: 1, sprintId: 1, status: 1 } — Compound for board queries
// { assigneeId: 1, status: 1 } — For "My Tasks" view
// { projectId: 1, createdAt: -1 }
// { sprintId: 1, storyPoints: 1 }
// { 'dependencies.taskId': 1 }
```

activity_logs (Time-Series Pattern)

```
{
  _id: ObjectId,
  workspaceId: ObjectId,
  userId: ObjectId,

  action: String,                    // 'task_created', 'task_moved', 'sprint_started',
                                     // 'comment_added', 'file_uploaded', 'ai_generated'
  entityType: String,                // 'task', 'sprint', 'project', 'workspace'
  entityId: ObjectId,

  metadata: Object,                  // Context-specific data

  // For analytics
  ipAddress: String,
  userAgent: String,

  createdAt: Date                    // TTL index: expire after 90 days (free) / 1 year (pro)
}

// Indexes
// { workspaceId: 1, createdAt: -1 } — For activity feeds
// { userId: 1, createdAt: -1 }      — For user activity
// { entityType: 1, entityId: 1 }    — For entity history
// { createdAt: 1 }                  — TTL index
```

6.2 Sharding Strategy

Shard Key: `workspaceId` on `tasks` and `activity_logs` collections.

Rationale:

- Workspaces are natural tenant boundaries
- Queries are almost always workspace-scoped
- Even distribution of data (assuming varied workspace sizes)
- Easy to isolate high-traffic workspaces to dedicated shards if needed

6.3 Data Lifecycle Policies

Collection	Retention Policy	Action
`activity_logs`	Free: 90 days, Pro: 1 year, Enterprise: 3 years	MongoDB TTL index

Table 5 – continued

Collection	Retention Policy	Action
`tasks` (deleted)	Soft delete → Hard delete after 30 days	Cron job
`attachments`	Orphaned files cleaned after task hard delete	S3 lifecycle + cron
`sprints` (completed)	Archived after 2 years	Move to cold storage

---

7. Authentication & Authorization

7.1 Authentication Strategies

Strategy	Implementation	User Flow
Email/Password	bcrypt (cost factor 12) + JWT	Register → Verify email → Login
Google OAuth	Passport.js Google Strategy	Click "Sign in with Google" → Consent → Auto-register/Login
GitHub OAuth	Passport.js GitHub Strategy	Click "Sign in with GitHub" → Consent → Auto-register/Login
Microsoft OAuth	Passport.js Microsoft Strategy	Enterprise SSO via Azure AD
Magic Links	Resend/SendGrid + signed tokens	Enter email → Click link → Auto-login (passwordless)

7.2 JWT Token Architecture

```
Token Lifecycle:

Client (Browser) → Access Token (15 min) → Redis Cache (Blacklist)
                          ↑                    ↓
                          | Token Expired    |
                          |                  |
                          ↓                  |
                  Refresh Token (7 days) → Redis Whitelist (Hash + User ID)
                  httpOnly cookie
```

7.3 Role-Based Access Control (RBAC)

Workspace Roles

Permission	Owner	Admin	Manager	Member
Delete Workspace	Yes	No	No	No
Manage Billing	Yes	No	No	No
Manage Members	Yes	Yes	No	No
Change Member Roles	Yes	Yes	No	No
Create/Edit Projects	Yes	Yes	Yes	No

Table 7 – continued

Permission	Owner	Admin	Manager	Member
Create/Edit Sprints	Yes	Yes	Yes	No
Edit Any Task	Yes	Yes	Yes	No
Edit Own Tasks	Yes	Yes	Yes	Yes
View Analytics	Yes	Yes	Yes	No
Use AI Features	Yes	Yes	Yes	Yes*
Manage Integrations	Yes	Yes	No	No

*Member gets 5 AI generations/month on free plan, unlimited on paid plans

7.4 Multi-Factor Authentication (MFA)

Implementation:

1. User enables MFA in settings
2. Server generates TOTP secret using `speakeasy`
3. QR code displayed for authenticator app scanning
4. User verifies by entering 6-digit code
5. Secret encrypted with AES-256 and stored in database
6. Future logins require TOTP code after password/OAuth

7.5 Security Features

Feature	Implementation
Password Policy	Min 12 chars, 1 uppercase, 1 number, 1 special char
Rate Limiting	express-rate-limit —100 req/15min per IP, 5 login attempts/ 15min
Account Lockout	5 failed logins = 15-minute lockout
Session Management	Redis-backed session store with TTL
Concurrent Session Limit	Max 5 active sessions per user
Password History	Last 5 passwords cannot be reused
Secure Headers	Helmet.js (CSP, HSTS, X-Frame-Options, etc.)
Audit Logging	All auth events logged to `activity_logs`

---

8. Payment Gateway

8.1 Pricing Model

Plan	Monthly Price	Annual Price	Features	Limits
Free	$0	$0	Basic kanban, 3 projects, manual sprint planning	5 members, no AI, no analytics
Pro	$12/user	$10/user	Unlimited projects, AI sprint planning, burndown charts, Slack integration, API access	50 users, 500 AI generations/ month
Enterprise	$25/user	$20/user	Everything in Pro + SSO/SAML, custom AI training, dedicated support, SLA 99.9%, advanced security	Unlimited users, unlimited AI, custom contracts

8.2 Stripe Integration Architecture

```
Billing Flow:

User (Client) → Selects Pro Plan → Enters Card
                    |
                    v
SprintSync Backend → Stripe API → Stripe Customer Created
                    |                    |
                    |                    v
                    |             Create Subscription (per-seat)
                    |                    |
                    |                    v
                    |             Invoice Created
                    |                    |
                    v                    v
Stripe Webhook (invoice.paid) → SprintSync (Provision Access)
```

8.3 Subscription Lifecycle

```
Key Stripe Events Handled:

1. checkout.session.completed
   → Create workspace subscription record
   → Send welcome email
   → Enable Pro features

2. invoice.paid
   → Extend subscription period
   → Update workspace quotas
   → Send receipt email
```

```
3. invoice.payment_failed
   → Mark subscription as "past_due"
   → Send payment failure email with retry link
   → Grace period: 3 days before feature downgrade

4. customer.subscription.updated
   → Handle plan changes (upgrade/downgrade)
   → Proration calculations for mid-cycle changes
   → Seat count changes (add/remove users)

5. customer.subscription.deleted
   → Downgrade to Free plan
   → Archive excess projects (read-only)
   → Send churn survey email
```

8.4 Seat Management

Per-Seat Billing Logic:

- Workspace admin invites members
- Each active member consumes one seat
- Billing automatically adjusts on next invoice (proration)
- Deactivated members free up seats at end of billing period
- Enterprise: Annual contracts with true-up billing

8.5 Webhook Security

```
Webhook handler with idempotency:

1. Verify Stripe signature
2. Check if event already processed (Redis: stripe:event:{id})
3. If processed, return 200 immediately
4. Process event
5. Mark as processed (24-hour TTL)
6. Return 200
```

---

9. AI Engine

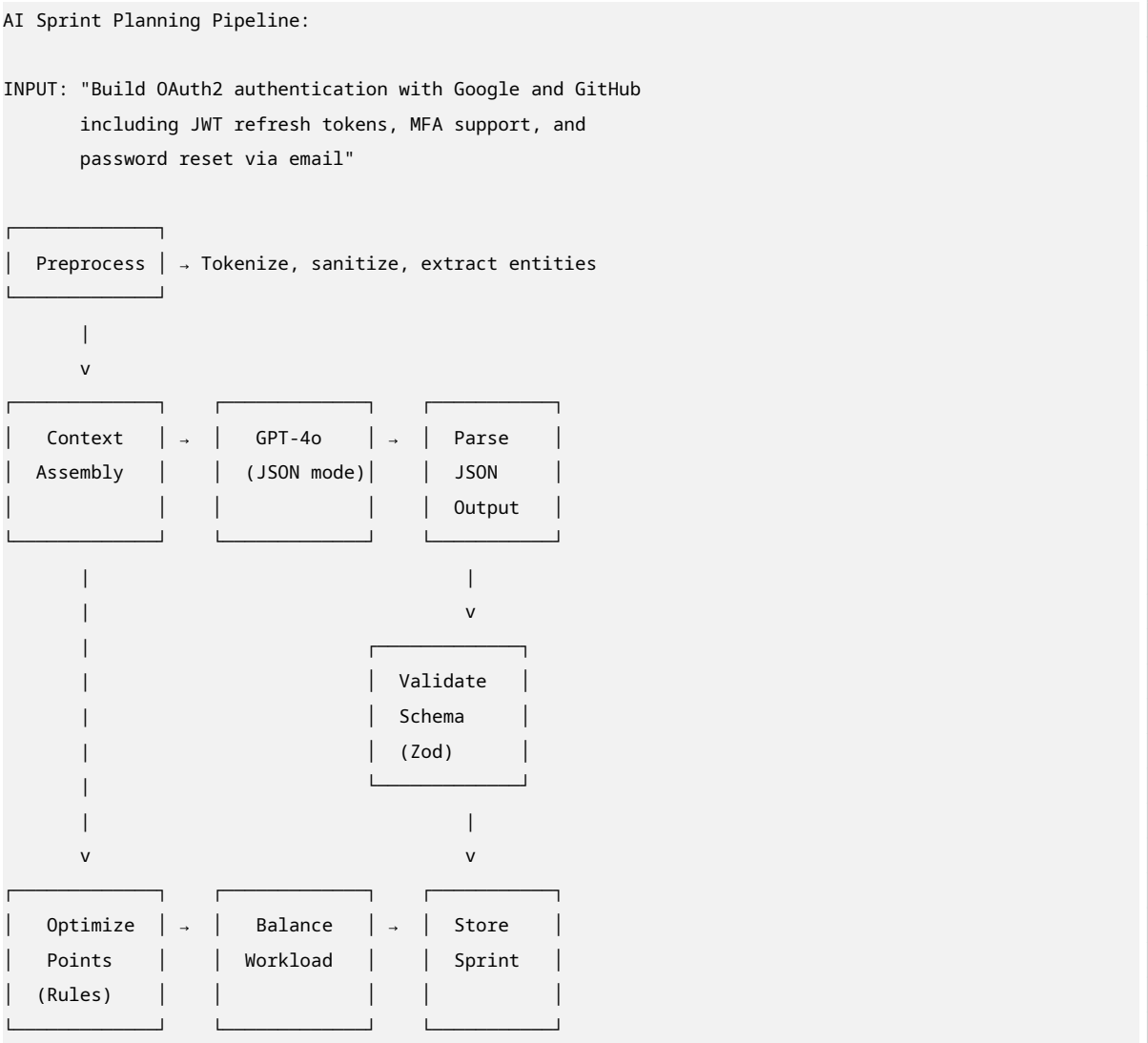
9.1 AI Feature Matrix

Feature	Model	Input	Output	Latency Target
Sprint Planning	GPT-4o	Product goal + team capacity + historical velocity	Sprint plan (tasks, points, assignments)	< 10s

Table 10 – continued

Feature	Model	Input	Output	Latency Target
Task Estimation	Fine-tuned GPT-3.5	Task description + historical similar tasks	Story points (1,2,3,5,8,13) + confidence	< 2s
Blocker Prediction	GPT-4o	Sprint tasks + dependencies + assignee workload	Blocker predictions + mitigation suggestions	< 5s
Standup Summary	GPT-4o	Task updates + commits + time logs	Formatted standup report	< 3s
Task Description Enhancement	GPT-3.5	Brief task title	Detailed acceptance criteria	< 3s

9.2 Sprint Planning AI Pipeline



```
OUTPUT:
{
  "sprintName": "Sprint 12: Auth System",
  "tasks": [
    { "title": "Set up OAuth2 base config", points: 3 },
    { "title": "Implement Google OAuth flow", points: 5 },
    { "title": "Implement GitHub OAuth flow", points: 5 },
    { "title": "JWT token generation & validation", points: 5 },
    { "title": "Refresh token rotation", points: 3 },
    { "title": "TOTP MFA setup", points: 5 },
    { "title": "Password reset email flow", points: 3 },
    { "title": "Integration tests", points: 5 }
  ],
  "totalPoints": 34,
  "recommendedDuration": 14,
  "predictedBlockers": [
    { "task": "TOTP MFA", "reason": "Requires security review" }
  ]
}
```

9.3 Fine-Tuning for Task Estimation

Training Data Format (JSONL):

```
{"messages": [{"role": "system", "content": "You are a story point estimator for software tasks. Respond
↪  with only a number: 1, 2, 3, 5, 8, or 13."}, {"role": "user", "content": "Task: Implement OAuth2
↪  login with Google and GitHub providers, including JWT refresh token rotation and MFA support"},
↪  {"role": "assistant", "content": "8"}]}
{"messages": [{"role": "system", "content": "You are a story point estimator for software tasks. Respond
↪  with only a number: 1, 2, 3, 5, 8, or 13."}, {"role": "user", "content": "Task: Fix CSS alignment
↪  bug on mobile navbar where hamburger menu overlaps logo on screens < 375px"}, {"role": "assistant",
↪  "content": "2"}]}
{"messages": [{"role": "system", "content": "You are a story point estimator for software tasks. Respond
↪  with only a number: 1, 2, 3, 5, 8, or 13."}, {"role": "user", "content": "Task: Design and implement
↪  real-time notification system with Socket.IO, Redis adapter, and email fallback for offline
↪  users"}, {"role": "assistant", "content": "13"}]}
```

Fine-Tuning Process:

1. Collect 500-1000 historical tasks with known actual completion times
2. Normalize descriptions (remove project-specific jargon)
3. Upload to OpenAI fine-tuning API
4. Train model for 3-4 epochs
5. Evaluate on holdout set (target: < 1 point average error)
6. Deploy model ID to production

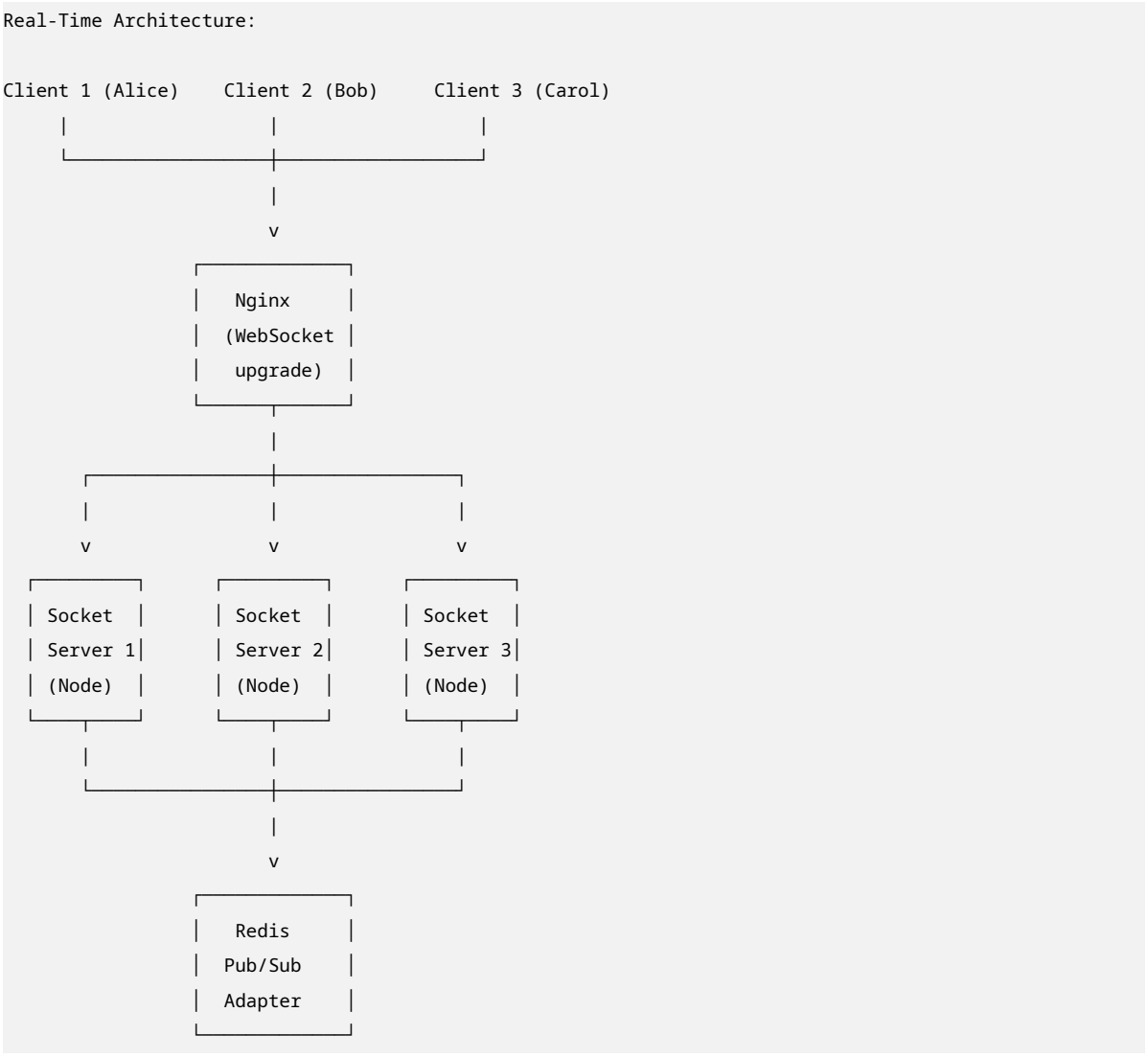
9.4 AI Cost Optimization

Strategy	Implementation	Savings
Caching	Redis cache for identical prompts (TTL: 1 hour)	30-40%
Model Selection	GPT-3.5 for estimation, GPT-4o only for planning	50-60%
Token Truncation	Limit input to 2000 tokens max	20-30%
Batch Processing	Queue AI jobs during off-peak hours	10-15%
Streaming	Stream planning output for perceived speed	UX improvement

---

10. Real-Time Collaboration

10.1 Socket.IO Architecture



10.2 Namespaces & Rooms

Namespace	Purpose	Rooms	Events
`/board`	Kanban board updates	`workspace:{id}`	`task:move`, `task:create`, `task:update`, `task:delete`
`/presence`	User activity status	`workspace:{id}`	`presence:update`, `presence:join`, `presence:leave`
`/sprint`	Sprint-specific updates	`sprint:{id}`	`sprint:start`, `sprint:complete`, `sprint:update`
`/notifications`	Real-time alerts	`user:{id}`	`notification:new`, `notification:read`

10.3 Event Protocol

```
// Standard event structure
interface SocketEvent<T = any> {
  event: string;           // 'task:move', 'comment:add', etc.
  payload: T;              // Event-specific data
  metadata: {
    timestamp: string;     // ISO 8601
    actorId: string;       // User who triggered
    workspaceId: string;
    clientId: string;      // Socket ID for deduplication
  };
}

// Example: Task moved on board
{
  event: 'task:move',
  payload: {
    taskId: 'task_123',
    fromStatus: 'todo',
    toStatus: 'in_progress',
    position: 2  // Index in new column
  },
  metadata: {
    timestamp: '2026-05-18T10:30:00Z',
    actorId: 'user_456',
    workspaceId: 'ws_789',
    clientId: 'socket_abc'
  }
}
```

10.4 Optimistic Updates with Conflict Resolution

```
Frontend: Optimistic Update

1. Optimistically update local state
```

```
2. Emit to server
3. Listen for confirmation or conflict
   - Confirmed → state is correct
   - Conflict → revert to server state, show warning
```

10.5 Presence Tracking

```
Server-side presence management:

userJoined(workspaceId, userId, socketId):
  - Store in Redis hash: presence:{workspaceId}
  - Broadcast to workspace: presence:join

userLeft(workspaceId, userId):
  - Remove from Redis hash
  - Broadcast to workspace: presence:leave

getOnlineUsers(workspaceId):
  - Fetch all from Redis hash
  - Return array of presence objects
```

---

11. API Design

11.1 RESTful API Structure

```
Base URL: https://api.sprintsync.app/v1

Authentication:
  POST   /auth/register
  POST   /auth/login
  POST   /auth/logout
  POST   /auth/refresh
  POST   /auth/forgot-password
  POST   /auth/reset-password
  POST   /auth/mfa/enable
  POST   /auth/mfa/verify
  POST   /auth/oauth/:provider

Workspaces:
  GET    /workspaces
  POST   /workspaces
  GET    /workspaces/:id
  PATCH  /workspaces/:id
  DELETE /workspaces/:id
  POST   /workspaces/:id/invite
  POST   /workspaces/:id/members/:userId/role
  DELETE /workspaces/:id/members/:userId

Projects:
```

```
  GET    /workspaces/:workspaceId/projects
  POST   /workspaces/:workspaceId/projects
  GET    /projects/:id
  PATCH  /projects/:id
  DELETE /projects/:id

Sprints:
  GET    /projects/:projectId/sprints
  POST   /projects/:projectId/sprints
  POST   /projects/:projectId/sprints/generate  # AI-generated sprint
  GET    /sprints/:id
  PATCH  /sprints/:id
  DELETE /sprints/:id
  POST   /sprints/:id/start
  POST   /sprints/:id/complete

Tasks:
  GET    /sprints/:sprintId/tasks
  GET    /projects/:projectId/tasks?status=backlog
  POST   /tasks
  GET    /tasks/:id
  PATCH  /tasks/:id
  DELETE /tasks/:id
  PATCH  /tasks/:id/status
  POST   /tasks/:id/assign
  POST   /tasks/:id/estimate  # AI estimation
  POST   /tasks/:id/dependencies
  POST   /tasks/:id/blockers
  POST   /tasks/:id/comments
  DELETE /tasks/:id/comments/:commentId

AI:
  POST   /ai/sprint-plan
  POST   /ai/task-estimate
  POST   /ai/predict-blockers
  POST   /ai/standup-summary

Analytics:
  GET    /workspaces/:workspaceId/analytics/velocity
  GET    /workspaces/:workspaceId/analytics/burndown?sprintId=...
  GET    /workspaces/:workspaceId/analytics/productivity
  GET    /workspaces/:workspaceId/analytics/dora

Billing:
  GET    /billing/plans
  POST   /billing/subscribe
  POST   /billing/upgrade
  POST   /billing/cancel
  GET    /billing/invoices
  POST   /billing/seats
  POST   /billing/webhooks/stripe
```

```
Integrations:
  POST   /integrations/slack/connect
  POST   /integrations/github/connect
  POST   /integrations/gitlab/connect
  GET    /integrations/status
```

11.2 API Standards

Aspect	Standard
Content-Type	`application/json`
Date Format	ISO 8601 (UTC)
Pagination	Cursor-based for activity logs, offset for lists
Rate Limiting	1000 req/hour per API key (Pro), 100 req/hour (Free)
Versioning	URL path (`/v1/`, `/v2/`)
Error Format	`{ "error": { "code": "...", "message": "...", "details": {} } }`
Filtering	`?status=todo,in_progress&assignee=user_123`
Sorting	`?sort=-createdAt` (descending)
Field Selection	`?fields=title,status,assignee`

11.3 GraphQL Endpoint (Enterprise)

For Enterprise customers requiring flexible data fetching:

```graphql
# Enterprise-only GraphQL API
query GetSprintWithTasks($sprintId: ID!) {
  sprint(id: $sprintId) {
    id
    name
    goal
    status
    totalPoints
    completedPoints
    velocity
    tasks {
      id
      title
      status
      storyPoints
      assignee {
        id
        name
        avatar
      }
      blockers {
        description
        predictedBy
      }
```

```
    }
    aiPredictions {
      predictedVelocity
      confidence
      suggestedBlockers
    }
  }
}
```

---

12. Frontend Architecture

12.1 Component Architecture (Atomic Design)

```
src/
├── components/
│   ├── atoms/              # Smallest units: Button, Input, Badge, Avatar
│   ├── molecules/          # Combinations: TaskCard, CommentItem, MemberRow
│   ├── organisms/          # Complex sections: KanbanColumn, SprintHeader, TaskModal
│   ├── templates/          # Page layouts: DashboardLayout, AuthLayout
│   └── pages/              # Route-level: BoardPage, SprintPage, AnalyticsPage
│
├── features/               # Domain-specific modules
│   ├── auth/
│   ├── board/
│   ├── sprint/
│   ├── task/
│   ├── analytics/
│   └── settings/
│
├── hooks/                  # Custom React hooks
│   ├── useSocket.ts
│   ├── useAuth.ts
│   ├── useWorkspace.ts
│   ├── useSprint.ts
│   ├── useTask.ts
│   └── useAI.ts
│
├── stores/                 # Zustand state slices
│   ├── auth.store.ts
│   ├── workspace.store.ts
│   ├── board.store.ts
│   └── ui.store.ts
│
├── services/               # API clients
│   ├── api.client.ts       # Axios instance with interceptors
│   ├── auth.service.ts
│   ├── workspace.service.ts
│   └── ai.service.ts
│
```

```
├── utils/                  # Helpers
│   ├── date.ts
│   ├── validators.ts
│   ├── constants.ts
│   └── formatters.ts
│
└── types/                  # TypeScript interfaces
    ├── auth.types.ts
    ├── workspace.types.ts
    ├── task.types.ts
    └── api.types.ts
```

12.2 State Management Strategy

State Type	Solution	Example
Server State	React Query	Tasks, sprints, workspaces (cached, background refetched)
Global UI State	Zustand	Sidebar open/closed, theme, modal visibility
Local Component State	useState/useReducer	Form inputs, local filters
Real-Time State	Socket.IO + Zustand	Live board updates, presence

12.3 Key Frontend Features

Kanban Board

- Drag-and-drop with `@dnd-kit/core` (accessible, performant)
- Virtualized rendering for boards with 500+ tasks (`react-window`)
- Inline editing (click title to edit, Enter to save)
- Quick actions (right-click context menu)

Sprint Planning View

- AI input panel (natural language prompt)
- Generated sprint preview (editable before creation)
- Team capacity visualization (hours/points per member)
- Conflict warnings (overloaded members, dependency cycles)

Analytics Dashboard

- Burndown chart with three lines: ideal, actual, predicted
- Velocity trend chart (last 10 sprints)
- Team productivity heatmap (commits vs. story points)
- DORA metrics cards with trend indicators

---

13. Background Jobs

13.1 Job Categories

Queue	Jobs	Priority	Schedule
`email`	Welcome emails, password resets, invoice receipts, digests	Normal	On-demand
`ai`	Sprint generation, task estimation, blocker prediction, standup summaries	High	On-demand + Scheduled
`notifications`	Slack messages, push notifications, in-app alerts	High	On-demand
`analytics`	Aggregate daily stats, compute velocity, generate reports	Low	Daily at 2 AM
`maintenance`	Clean expired sessions, archive old data, S3 cleanup	Low	Hourly
`billing`	Invoice generation, payment retries, usage calculations	Critical	On-demand + Daily

13.2 Job Implementation

```
AI Job Worker:

Queue: 'ai'
Connection: Redis
Default Options:
  - attempts: 3
  - backoff: exponential, 5s delay
  - removeOnComplete: 100
  - removeOnFail: 50

Worker Logic:
  switch (job.type):
    'generate-sprint': return generateSprintPlan(data)
    'estimate-task': return estimateTask(data)
    'predict-blockers': return predictBlockers(data)
    'generate-standup': return generateStandupSummary(data)
    default: throw Error('Unknown AI job type')
```

```
Concurrency: 5 jobs simultaneously
Rate Limiter: 50 jobs per minute

Error Handling:
  - Log failed jobs with context
  - Alert on-call if all attempts exhausted
```

---

14. Third-Party Integrations

14.1 Slack Integration

Features:

- Sprint start/completion notifications
- Task assignment alerts
- Daily standup summaries
- `/sprintsync` slash commands
- Interactive buttons (approve tasks, view details)

Implementation:

- OAuth 2.0 app installation
- Incoming webhooks for notifications
- Slack API for interactive components
- Event subscriptions for user actions

14.2 GitHub/GitLab Integration

Features:

- Link commits to tasks ("Closes TASK-123")
- PR status in task details
- Branch creation from task
- Auto-move task to "In Review" when PR opened
- Auto-move to "Done" when PR merged

Implementation:

- GitHub App (webhook-based)
- GitLab System Hooks
- GraphQL API for rich data fetching

14.3 Calendar Integration

Features:

- Sprint start/end dates to Google/Outlook calendar
- Task due dates as calendar events
- Sprint planning meeting scheduling

---

15. DevOps & Deployment

15.1 Docker Configuration

```
# Multi-stage build for API
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS production
WORKDIR /app
RUN apk add --no-cache dumb-init
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package.json ./
EXPOSE 3000
USER node
CMD ["dumb-init", "node", "dist/app.js"]
```

15.2 CI/CD Pipeline (GitHub Actions)

```
SprintSync CI/CD:

Triggers:
  - Push to main/develop
  - Pull request to main

Jobs:
  lint-and-test:
    - Checkout code
    - Setup Node.js 20
    - Install dependencies
    - Run linter
    - Run unit tests with coverage
    - Run integration tests
    - Upload coverage to Codecov

  build-and-deploy:
```

```
Needs: lint-and-test
If: branch == main
Steps:
  - Build Docker images (API, Web, Nginx)
  - Push to container registry
  - Deploy to staging (Kubernetes)
  - Run smoke tests
  - Deploy to production (Kubernetes)
```

15.3 Monitoring & Alerting

Tool	Purpose	Metrics
Prometheus	Metrics collection	Request rate, latency, error rate, DB connections, Redis memory
Grafana	Visualization	Dashboards for API health, real-time connections, AI job queue depth
Sentry	Error tracking	Uncaught exceptions, AI model failures, payment errors
LogRocket	Frontend monitoring	User session replay, frontend errors, performance metrics
UptimeRobot	External monitoring	API availability, SSL certificate expiry

15.4 Scaling Strategy

Traffic Level	Architecture	Scaling Action
0-1K users	Single server + MongoDB replica	Vertical scaling (bigger instances)
1K-10K users	3 API servers + 2 Socket servers	Horizontal scaling, CDN for static assets
10K-100K users	Kubernetes auto-scaling + MongoDB sharding	Shard by workspaceId, Redis Cluster
100K+ users	Microservices extraction	Separate AI service, dedicated analytics pipeline

---

16. Security Considerations

16.1 Data Protection

Layer	Implementation
Encryption at Rest	MongoDB encryption (AES-256), S3 SSE-S3
Encryption in Transit	TLS 1.3 for all communications
Field-Level Encryption	PII (emails, names) encrypted in database
API Keys	Hashed with bcrypt, never logged
File Uploads	Virus scanning (ClamAV), type validation, size limits

16.2 Compliance Roadmap

Standard	Priority	Implementation
SOC 2 Type I	High	Audit logging, access controls, incident response
SOC 2 Type II	Medium	6-month observation period after Type I
GDPR	High	Data deletion, export, consent management, DPA
CCPA	Medium	Consumer rights fulfillment
ISO 27001	Low	Enterprise customer requirement

16.3 Security Checklist

- Input validation on all API endpoints (Zod schemas)
- SQL/NoSQL injection prevention (Mongoose parameterized queries)
- XSS protection (React auto-escaping + CSP headers)
- CSRF protection (SameSite cookies + anti-CSRF tokens)
- Rate limiting per IP and per user
- Dependency vulnerability scanning (Snyk, Dependabot)
- Secrets management (AWS Secrets Manager, never in code)
- Penetration testing (quarterly)
- Security headers (Helmet.js: HSTS, X-Frame-Options, etc.)

---

17. Implementation Roadmap

Phase 1: Foundation (Weeks 1-2)

Goal: Working authentication and workspace management

Task	Deliverable
Project setup	Monorepo with Turborepo, Docker, linting
Database design	MongoDB schemas, indexes, seed scripts
Auth system	Register, login, JWT, OAuth, MFA
Workspace CRUD	Create, invite, manage members
Frontend shell	React + Vite + Tailwind + routing

Phase 2: Core PM Features (Weeks 3-5)

Goal: Functional kanban board and sprint management

Task	Deliverable
Project management	Create, edit, archive projects
Sprint CRUD	Planning, active, completed sprints
Task CRUD	Full task lifecycle with rich text
Kanban board	Drag-and-drop, columns, filters

Table 21 – continued

Task	Deliverable
Basic notifications	Email alerts for assignments

Phase 3: Real-Time Collaboration (Weeks 6-7)

Goal: Live board updates and presence tracking

Task	Deliverable
Socket.IO setup	Server + client with Redis adapter
Live board	Real-time task moves, instant sync
Presence system	Online status, typing indicators
Comments	Real-time comment threads
Optimistic updates	Instant UI feedback with conflict handling

Phase 4: AI Engine (Weeks 8-9)

Goal: AI-powered sprint planning and estimation

Task	Deliverable
OpenAI integration	API client, prompt engineering
Sprint generation	Natural language → sprint plan
Task estimation	Fine-tuned model for story points
Blocker prediction	Dependency analysis + AI insights
Standup summaries	Automated daily reports

Phase 5: Billing & Integrations (Week 10)

Goal: Monetization and ecosystem connections

Task	Deliverable
Stripe integration	Subscriptions, invoices, webhooks
Pricing page	Plan comparison, checkout flow
Slack integration	Notifications, slash commands
GitHub integration	Commit linking, PR status
API access	REST API + documentation

Phase 6: Analytics & Polish (Weeks 11-12)

Goal: Insights and production readiness

Task	Deliverable
Burndown charts	Ideal vs. actual vs. predicted
Velocity tracking	Historical trends, team comparison
DORA metrics	Deployment frequency, lead time, MTTR, CFR

Table 25 – continued

Task	Deliverable
Performance optimization	Lazy loading, code splitting, image optimization
Testing	E2E tests (Playwright), load testing (k6)
Documentation	API docs, user guides, admin manual

---

18. Monetization Strategy

18.1 Revenue Streams

Stream	Description	Expected Revenue %
Subscription Revenue	Monthly/annual per-seat billing	85%
AI Usage Overages	Additional AI generations beyond plan limits	10%
Enterprise Add-ons	Custom integrations, dedicated support, SLA	5%

18.2 Customer Acquisition Strategy

Channel	Tactic	CAC Target
Product-Led Growth	Free tier with viral features (team invites)	$0
Content Marketing	Engineering blog, AI sprint planning guides	$50
Open Source	Open-source CLI tools and GitHub Actions	$30
Developer Communities	Hacker News, Reddit, Dev.to, Indie Hackers	$20
Integration Marketplace	Slack App Directory, GitHub Marketplace	$40
Paid Ads	Google Ads (“Jira alternative”), LinkedIn	$150

18.3 Unit Economics (Pro Plan)

```
Monthly Revenue per User: $12
Monthly Cost per User:
  - Infrastructure: $2.50
  - AI API costs: $1.50
  - Payment processing (Stripe): $0.36
  - Customer support: $0.50
  - Miscellaneous: $0.50
Total Cost per User: $5.36
```

```
Gross Margin: 55%
LTV (24-month average): $288
CAC Payback Period: 3-4 months
```

---

19. Competitive Analysis

19.1 Feature Comparison Matrix

Feature	SprintSync	Jira	Linear	Notion	Trello
AI Sprint Planning	Yes	No	No	No	No
Predictive Blockers	Yes	No	No	No	No
Real-Time Board	Yes	Partial	Yes	No	No
Live Cursors	Yes	No	No	No	No
Auto Standups	Yes	No	No	No	No
Fine-Tuned Estimation	Yes	No	No	No	No
Transparent Pricing	Yes	No	Yes	Yes	Yes
Per-Seat Billing	Yes	Yes	Yes	Yes	Yes
SSO/SAML	Enterprise	Enterprise	Enterprise	Enterprise	Enterprise
API Access	Pro+	Enterprise	Pro+	Enterprise	Power-Up
DORA Metrics	Yes	Partial	No	No	No

19.2 Competitive Positioning

```
Complexity vs. Intelligence Matrix:

High Complexity + Low Intelligence: Jira, Azure DevOps
Low Complexity + Low Intelligence: Trello, Asana
High Complexity + High Intelligence: Linear (partial)
Low Complexity + High Intelligence: SPRINTSYNC (target)
```

---

20. Future Enhancements

20.1 Near-Term (6 Months Post-Launch)

Feature	Description	Impact
Mobile App	React Native iOS/Android app	+30% user engagement
Offline Mode	PWA with local-first sync	Remote team retention
Advanced Analytics	Cohort analysis, churn prediction	Enterprise sales
White-Label	Custom branding for agencies	New revenue stream

Table 29 – continued

Feature	Description	Impact
AI Retrospectives	Automated sprint retrospective generation	Team efficiency

20.2 Long-Term (12-24 Months)

Feature	Description	Impact
AI Product Manager	Autonomous backlog prioritization, roadmap suggestions	Market differentiation
Voice Commands	"SprintSync, create a task for OAuth setup, 5 points, assign to Alice"	Accessibility
Code Intelligence	Auto-link tasks to code changes, suggest test coverage	Developer productivity
Multi-Workspace Analytics	Portfolio-level insights for executives	Enterprise expansion
Marketplace	Third-party integrations and custom workflows	Ecosystem lock-in

20.3 Technical Debt & Evolution

Phase	Action	Trigger
Microservices	Extract AI service, billing service, notification service	100K+ MAU
GraphQL Federation	Unified GraphQL gateway across services	5+ microservices
Event Sourcing	Full event sourcing for audit trails	Enterprise compliance
ML Pipeline	Custom model training infrastructure	10M+ AI requests/month
Edge Computing	CDN-level real-time sync for global teams	50% users outside US

Appendix A: Development Environment Setup

Prerequisites

- Node.js 20+
- Docker & Docker Compose
- MongoDB 7 (local or Atlas)
- Redis 7
- Stripe CLI (for webhook testing)
- OpenAI API key

Quick Start

```
# Clone repository
git clone https://github.com/your-org/sprintsync.git
cd sprintsync

# Install dependencies
npm install

# Setup environment variables
cp .env.example .env
# Edit .env with your API keys

# Start infrastructure
docker-compose up -d mongodb redis

# Run database migrations
npm run db:migrate

# Seed development data
npm run db:seed

# Start development servers
npm run dev

# Access application
# Web: http://localhost:5173
# API: http://localhost:3000
```

Appendix B: Testing Strategy

Test Pyramid

Layer	Tool	Coverage Target
Unit Tests	Jest	80%
Integration Tests	Supertest + Testcontainers	60%
E2E Tests	Playwright	40%
Load Tests	k6	Key flows
Contract Tests	Pact	API boundaries

Critical Test Scenarios

1. Authentication: Register → Verify → Login → Refresh → Logout
2. Sprint Planning: Create workspace → Generate AI sprint → Verify tasks → Start sprint
3. Real-Time: Open board in 2 browsers → Move task → Verify sync in < 500ms
4. Billing: Subscribe → Process payment → Verify access → Cancel → Verify downgrade
5. AI Accuracy: 100 test prompts → Verify JSON schema compliance → Check point totals

---

Appendix C: API Rate Limits

Plan	Requests/Minute	AI Generations/Month	WebSocket Connections
Free	60	0	1
Pro	300	500	5
Enterprise	1000	Unlimited	20

Document Version: 1.0 Last Updated: May 2026 Author: SprintSync Engineering Team